LAPORAN PRAKTIKUM MACHINE LEARNING
LINEAR REGRESSION



Oleh:

Nama : Elsadanika Maura Sitorus

NIM : 211344013

Tanggal Praktikum : 24 Oktober 2023

Tanggal Pengumpulan Laporan : 27 Oktober 2023

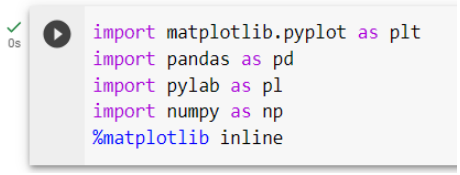
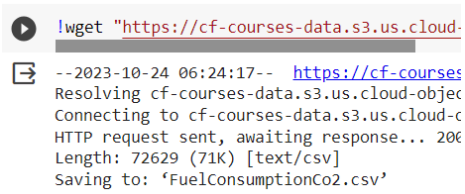
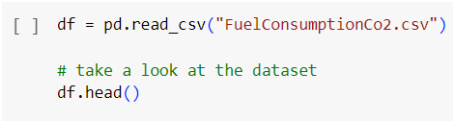
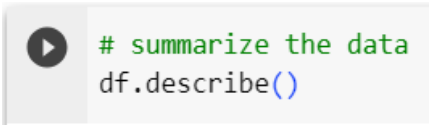
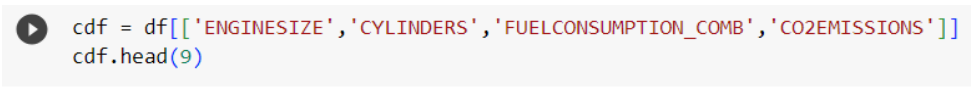
PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

OKTOBER 2023

1. Linear Regression

a) Kode Program dan Penjelasannya

Kode Program	Fungsi
	Mengimport library yang dibutuhkan dalam menjalankan program yang kemudian disimpan sebagai nama lain yang lebih singkat.
	Mendownload dataset dari IBM Object Storage melalui perintah !wget yang disertai dengan link data dari IBM.
FuelConsumptionCo2.csv:	Melalui perintah !wget disertai link, maka akan terdownload dataset tipe .csv yang mengandung model-spesific dan estimasi output.
	- df merupakan variabel baru - pd merupakan library pandas - simbol dot (.) menandakan penggunaan method pada library package - read_csv merupakan method untuk membaca data yang ada pada file terdownload yaitu FuelConsumptionCo2.csv - method head digunakan untuk melihat sebagian data awal dalam dataframe.
	digunakan method describe untuk menghasilkan ringkasan statistik deskriptif dari dataframe.
 baris pertama : untuk memilih kolom tertentu dari df baris kedua : untuk menampilkan sembilan baris pertama dari subset dataframe cdf yang dibuat pada baris pertama.	

```
[ ]  viz = cdf[['CYLINDERS','ENGINESIZE','CO2EMISSIONS','FUELCONSUMPTION_COMB']]
     viz.hist()
     plt.show()
```

baris 1 : membuat subset baru dari dataframe df disimpan dalam viz
baris 2 : memanggil method hist() pada viz untuk membuat histogram
baris 3 : untuk menampilkan gambar histogram setiap kolom viz

```
     plt.scatter(cdf.FUELCONSUMPTION_COMB, cdf.CO2EMISSIONS,  color='blue')
     plt.xlabel("FUELCONSUMPTION_COMB")
     plt.ylabel("Emission")
     plt.show()
```

baris 1 : membuat plot scatter untuk visualisasi hubungan dua variabel
baris 4 : menampilkan plot scatter

```
[ ]  msk = np.random.rand(len(df)) < 0.8
     train = cdf[msk]
     test = cdf[~msk]
```

- np adalah library numpy
- diambil 80% train set dan 20% test set dilihat dari jumlah keseluruhan data pada df
- Ketika df = 1067, maka train = 854 dan test = 213 data.

```
     from sklearn import linear_model
     regr = linear_model.LinearRegression()
     train_x = np.asanyarray(train[['ENGINESIZE']])
     train_y = np.asanyarray(train[['CO2EMISSIONS']])
     regr.fit(train_x, train_y)
     # The coefficients
     print ('Coefficients: ', regr.coef_) #kemiringan/slope
     print ('Intercept: ',regr.intercept_) #titik awal di y

     Coefficients:  [[39.21775516]]
     Intercept:  [125.35298661]
```

baris 1 : mengimpor library linear_model dari sklearn
baris 2 : membuat objek dari class LinearRegression()
baris 3 : mengambil kolom ENGINESIZE dari data train dan dikonversi menjadi array numpy
baris 5 : melatih model, model akan mempelajari hubungan train_x dan train_y untuk mencari coefficient (kemiringan) dan intercept (titik awal sumbu y).

```
[ ]  from sklearn.metrics import r2_score

     test_x = np.asanyarray(test[['ENGINESIZE']])
     test_y = np.asanyarray(test[['CO2EMISSIONS']])
     test_y_ = regr.predict(test_x)

     print("Mean absolute error: %.2f" % np.mean(np.absolute(test_y_ - test_y)))
     print("Residual sum of squares (MSE): %.2f" % np.mean((test_y_ - test_y) ** 2))
     print("R2-score: %.2f" % r2_score(test_y , test_y_) )

     Mean absolute error: 22.83
     Residual sum of squares (MSE): 882.17
     R2-score: 0.73
```

baris 1 : mengambil kolom ENGINESIZE dari test set dan dikonversi menjadi numpy array untuk menguji model
baris 3 : menggunakan model regresi linier sebelumnya (“regr”) untuk membuat prediksi

b) Input dan Output Program Setelah Dijalankan

Input

```
df = pd.read_csv("FuelConsumptionCo2.csv")

# take a look at the dataset
df.head()
```

Output

	MODELYEAR	MAKE	MODEL	VEHICLECLASS	ENGINESIZE	CYLINDERS
0	2014	ACURA	ILX	COMPACT	2.0	4
1	2014	ACURA	ILX	COMPACT	2.4	4
2	2014	ACURA	ILX HYBRID	COMPACT	1.5	4
3	2014	ACURA	MDX 4WD	SUV - SMALL	3.5	6
4	2014	ACURA	RDX AWD	SUV - SMALL	3.5	6

Input

```
# summarize the data
df.describe()
```

Output

	MODELYEAR	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_CITY
count	1067.0	1067.000000	1067.000000	1067.000000
mean	2014.0	3.346298	5.794752	13.296532
std	0.0	1.415895	1.797447	4.101253
min	2014.0	1.000000	3.000000	4.600000
25%	2014.0	2.000000	4.000000	10.250000
50%	2014.0	3.400000	6.000000	12.600000
75%	2014.0	4.300000	8.000000	15.550000
max	2014.0	8.400000	12.000000	30.200000

Input

```
cdf = df[['ENGINESIZE','CYLINDERS','FUELCONSUMPTION_COMB','CO2EMISSIONS']]
cdf.head(9)
```

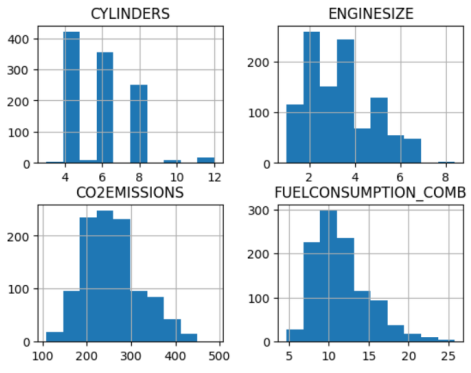
Output

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

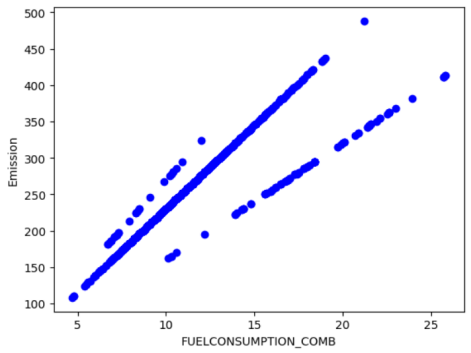
Input

```
viz = cdf[['CYLINDERS','ENGINESIZE','CO2EMISSIONS','FUELCONSUMPTION_COMB']]
viz.hist()
plt.show()
```

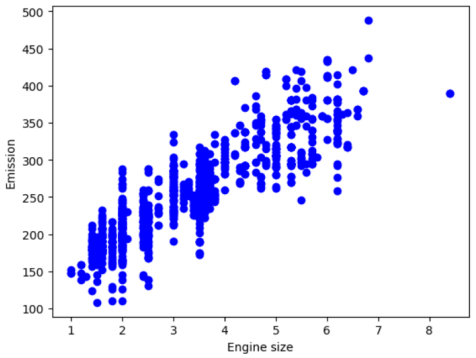
Output



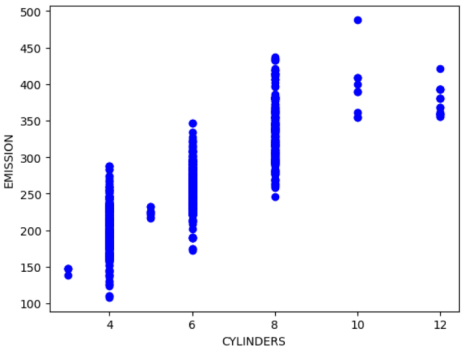
```
plt.scatter(cdf.FUELCONSUMPTION_COMB, cdf.CO2EMISSIONS,  color='blue')
plt.xlabel("FUELCONSUMPTION_COMB")
plt.ylabel("Emission")
plt.show()
```



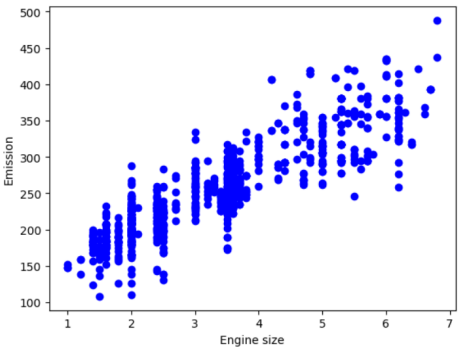

```
plt.scatter(cdf.ENGINESIZE, cdf.CO2EMISSIONS,  color='blue')
plt.xlabel("Engine size")
plt.ylabel("Emission")
plt.show()
```




```
# write your code here
plt.scatter(cdf.CYLINDERS, cdf.CO2EMISSIONS, color='blue')
plt.xlabel("CYLINDERS")
plt.ylabel("EMISSION")
plt.show()
```




```
plt.scatter(train.ENGINESIZE, train.CO2EMISSIONS,  color='blue')
plt.xlabel("Engine size")
plt.ylabel("Emission")
plt.show()
```




```
from sklearn import linear_model
regr = linear_model.LinearRegression()
train_x = np.asanyarray(train[['ENGINESIZE']])
train_y = np.asanyarray(train[['CO2EMISSIONS']])
regr.fit(train_x, train_y)
# The coefficients
print ('Coefficients: ', regr.coef_) #kemiringan/slope
print ('Intercept: ',regr.intercept_) #titik awal di y
```

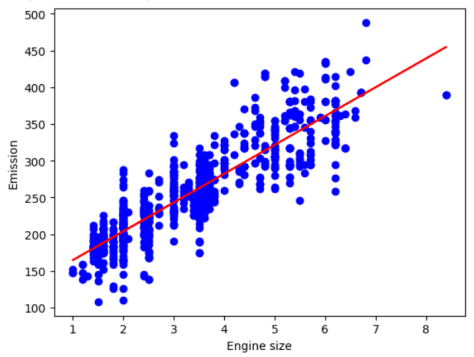
```
Coefficients:  [[39.21775516]]
Intercept:  [125.35298661]
```

<table>
<tr><td>

```
plt.scatter(train.ENGINESIZE, train.CO2EMISSIONS,  color='blue')
plt.plot(train_x, regr.coef_[0][0]*train_x + regr.intercept_[0], '-r')
plt.xlabel("Engine size")
plt.ylabel("Emission")
```

</td><td>

```
Text(0, 0.5, 'Emission')
```



</td></tr>
<tr><td>

```
from sklearn.metrics import r2_score

test_x = np.asanyarray(test[['ENGINESIZE']])
test_y = np.asanyarray(test[['CO2EMISSIONS']])
test_y_ = regr.predict(test_x)

print("Mean absolute error: %.2f" % np.mean(np.absolute(test_y_ - test_y)))
print("Residual sum of squares (MSE): %.2f" % np.mean((test_y_ - test_y) ** 2))
print("R2-score: %.2f" % r2_score(test_y , test_y_) )
```

</td><td>

```
Mean absolute error: 22.83
Residual sum of squares (MSE): 882.17
R2-score: 0.73
```

</td></tr>
<tr><td colspan="2">Exercise</td></tr>
<tr><td>

```
[14] train_x = train[["FUELCONSUMPTION_COMB"]]

     test_x = test[["FUELCONSUMPTION_COMB"]] #
```

</td><td>no output</td></tr>
<tr><td>

```
regr = linear_model.LinearRegression()
regr.fit(train_x, train_y)
#ADD CODE
```

</td><td>

```
LinearRegression
LinearRegression()
```

</td></tr>
<tr><td>

```
predictions = regr.predict(test_x)
```

</td><td>no output</td></tr>
<tr><td>

```
#ADD CODE
print("Mean Absolute Error: %.2f" %np.mean(np.absolute(predictions - test_y)))
```

</td><td>

```
Mean Absolute Error: 18.09
```

</td></tr>
</table>

c) Analisis

Linear regression merupakan teknik analisis data yang memprediksi nilai data yang tidak diketahui menggunakan nilai data lain yang terkait dan diketahui. Pada program pada praktikum, dilakukan analisis prediksi linear regresi antara ukuran mesin atau ENGINESIZE dengan emisi karbon dioksida atau CO2EMISSIONS. Dari dataset yang telah didownload melalui perintah !wget, terdapat total 1067 data yang kemudian dibagi menjadi 2 bagian yaitu train set dan test set. Train set adalah data-data yang digunakan untuk melatih mesin memprediksi, digunakan 80% dari total data. Sedangkan test set adalah data-data yang digunakan untuk menguji mesin, digunakan sebanyak 20% dari

total data. Pemisahan data ini dilakukan untuk memeriksa kinerja dari model yang telah dibuat pada data yang belum pernah diketahui sebelumnya.

Setelah pemisahan data, dibuat program untuk memahami hubungan antara variabel independen yaitu ENGINESIZE dengan variabel dependen yaitu CO2EMISSIONS. Digunakan library scikit-learn yang disingkat sebagai sklearn. Melalui program ini, maka akan didapatkan garis regresi yang paling baik dan paling sesuai dengan data train. Perintah kunci untuk menjalankan program ini adalah from sklearn import linear_model.

Untuk membentuk garis regresi, perlu diketahui nilai intercept dan koefisiennya sehingga dapat ditentukan titik mulai garis pada sumbu vertical, dan nilai kemiringan dari garis tersebut. Setelah model linear regression dilatih melalui train data, maka selanjutnya program akan mencetak nilai koefisien dan intercept. Sebelum mencetak, dibutuhkan perintah bahwa digunakan class LinearRegression() dari library linear_model yang diambil dari library sklearn. Lalu diambil data array untuk enginesize data train yang dikonversi ke library np dan disimpan pada train_x, begitupun untuk co2emission yang disimpan pada train_y. Untuk membentuk garis, maka digunakan perintah pemanggilan method fit untuk train_x dan train_y, sehingga nilai coefficient dan intercept dapat dicetak.

Selanjutnya, program diuji melalui penggunaan test atau pengujian dataset, yaitu data-data yang tidak digunakan selama training model. Setelah pengujian, langkah terakhir yang dilakukan adalah evaluasi kinerja. Dilakukan evaluasi melalui penggunaan library sklearn.metrics dengan methodnya r2_score. Lalu diambil data array untuk enginesize data test yang dikonversi ke library np dan disimpan pada test_x, begitupun untuk co2emission yang disimpan pada test_y. Untuk mengetahi nilai-nilai errornya, maka digunakan perintah test_y = regr.predict(test_x) sehingga dapat diketahui nilai error yang terjadi pada output y oleh input x. r2-score mengukur sejauh mana variasi dalam data aktual dapat dijelaskan oleh model. Nilai r2-score adalah 0 hingga 1, semakin tinggi nilainya maka semakin baik model dalam menjelaskan variasi data.

LAPORAN PRAKTIKUM MACHINE LEARNING
CLASSIFICATION LOGISTIC REGRESSION



Oleh:

Nama : Elsadanika Maura Sitorus

NIM : 211344013

Tanggal Praktikum : 24 Oktober 2023

Tanggal Pengumpulan Laporan : 27 Oktober 2023

PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

OKTOBER 2023

2. Classification Logistic Regression

a) Kode Program dan Penjelasannya

Kode Program	Fungsi

`import pandas as pd`
`import pylab as pl`
`import numpy as np`
`import scipy.optimize as opt`
`from sklearn import preprocessing`
`%matplotlib inline`
`import matplotlib.pyplot as plt`
	Mengimport library yang dibutuhkan dalam menjalankan program yang kemudian disimpan sebagai nama lain yang lebih singkat.

`!wget "https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-`	Mendownload dataset dari IBM Object Storage melalui perintah !wget yang disertai dengan link data dari IBM.
`churn_df = pd.read_csv("ChurnData.csv")` `churn_df.head()`	baris 1 : membaca data dari file csv bernama ChurnData.csv lalu dimuat kedalam dataframe churn.df dengan library pandas. baris 2 : menampilkan 5 baris pertama dari data terbaca.

`churn_df = churn_df[['tenure','age','address','income','ed','employ','equip','callcard','wireless','churn']]`
`churn_df['churn'] = churn_df['churn'].astype('int')`
`churn_df.head()`
	baris 1 : modifikasi dataframe untuk menampilkan kolom tertemtu.
	baris 2 : mengubah tipe data kolom churn menjadi integer

`# write your code here` `churn_df.shape`	memanggil method shape sehhingga mengetahui bentuk atau dimensi dataframe.
`X = np.asarray(churn_df[['tenure','age','address','income','ed','employ','equip']])` `X[0:5]`	baris 1 : membuat variabel x untuk menyimpan data array dari library np. baris 2 : menampilkan 5 baris pertama dari array x

`from sklearn import preprocessing`
`X = preprocessing.StandardScaler().fit(X).transform(X)`
`X[0:5]`
	baris 1 : import library preprocessing dari sklearn
	baris 2 : membuat objek StandardScaler() untuk nilai normalisasi, lalu menghitung parameter transformasi array x.

`from sklearn.model_selection import train_test_split`
`X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=4)`
`print ('Train set:', X_train.shape,  y_train.shape)`
`print ('Test set:', X_test.shape,  y_test.shape)`
	baris 2 : membagi dataset menjadi 4 variabel dengan data untuk test_size

sebanyak 20%.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
LR = LogisticRegression(C=0.01, solver='liblinear').fit(X_train,y_train)
LR
```

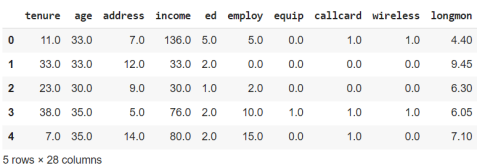
baris 3 : membuat objek LogisticRegression dengan parameter C (nilai regularisasi yang akan diterapkan) dan solver (algoritma yang digunakan), memanggil method .fit() untuk melatih model dengan test set.
baris 4 : variabel LR akan berisi model regresi logistik yang sudah dilatih untuk memprediksi data baru.

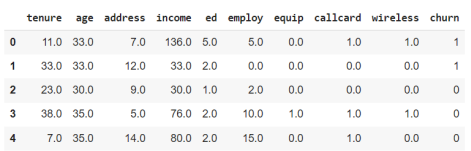
`yhat = LR.predict(X_test)` `yhat`	baris 1 : memanggil method predict pada LR untuk memprediksi X_test baris 2 : akan berisi array hasil prediksi model untuk setiap dataset pengujian y_test
`yhat_prob = LR.predict_proba(X_test)` `yhat_prob`	baris 1 : untuk menghitug probabilitas prediksi. baris 2 : akan berisi array hasil probabilitas prediksi.
`from sklearn.metrics import jaccard_score` `jaccard_score(y_test, yhat,pos_label=0)`	baris 2 : menghitung skor jaccard antara y_test dengan yhat. pos_label adalah parameter penentuk kelas yang dianggap sebagai label positif.

```
plt.imshow(cm, interpolation='nearest', cmap=cmap)
```

membuat plot confussion matrix dengan tambahan label pada sumbu x dan y

b) Input dan Output Program Setelah Dijalankan

Input	Output
`churn_df = pd.read_csv("ChurnData.csv")` `churn_df.head()`	

`churn_df = churn_df[['tenure','age','address','income','ed','emp`
`churn_df['churn'] = churn_df['churn'].astype('int')`
`churn_df.head()`
	

`# write your code here` `churn_df.shape`	`(200, 10)`

	tenure	age	address	income	ed	employ	equip	callcard	wireless	longmon
0	11.0	33.0	7.0	136.0	5.0	5.0	0.0	1.0	1.0	4.40
1	33.0	33.0	12.0	33.0	2.0	0.0	0.0	0.0	0.0	9.45
2	23.0	30.0	9.0	30.0	1.0	2.0	0.0	0.0	0.0	6.30
3	38.0	35.0	5.0	76.0	2.0	10.0	1.0	1.0	1.0	6.05
4	7.0	35.0	14.0	80.0	2.0	15.0	0.0	1.0	0.0	7.10

5 rows × 28 columns

	tenure	age	address	income	ed	employ	equip	callcard	wireless	churn
0	11.0	33.0	7.0	136.0	5.0	5.0	0.0	1.0	1.0	1
1	33.0	33.0	12.0	33.0	2.0	0.0	0.0	0.0	0.0	1
2	23.0	30.0	9.0	30.0	1.0	2.0	0.0	0.0	0.0	0
3	38.0	35.0	5.0	76.0	2.0	10.0	1.0	1.0	1.0	0
4	7.0	35.0	14.0	80.0	2.0	15.0	0.0	1.0	0.0	0

```
X = np.asarray(churn_df[['tenure','age','address','income','ed','employ','equip']])
X[0:5]
```

```
array([[ 11.,  33.,   7., 136.,   5.,   5.,   0.],
       [ 33.,  33.,  12.,  33.,   2.,   0.,   0.],
       [ 23.,  30.,   9.,  30.,   1.,   2.,   0.],
       [ 38.,  35.,   5.,  76.,   2.,  10.,   1.],
       [  7.,  35.,  14.,  80.,   2.,  15.,   0.]])
```

```
y = np.asarray(churn_df['churn'])
y [0:5]
```

```
array([1, 1, 0, 0, 0])
```

```
from sklearn import preprocessing
X = preprocessing.StandardScaler().fit(X).transform(X)
X[0:5]
```

```
array([[-1.13518441, -0.62595491, -0.4588971 ,  0.4751423 ,  1.6961288 ,
        -0.58477841, -0.85972695],
       [-0.11604313, -0.62595491,  0.03454064, -0.32886061, -0.6433592 ,
        -1.14437497, -0.85972695],
       [-0.57928917, -0.85594447, -0.261522  , -0.35227817, -1.42318853,
        -0.92053635, -0.85972695],
       [ 0.11557989, -0.47262854, -0.65627219,  0.00679109, -0.6433592 ,
        -0.02518185,  1.16316   ],
       [-1.32048283, -0.47262854,  0.23191574,  0.03801451, -0.6433592 ,
         0.53441472, -0.85972695]])
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=4)
print ('Train set:', X_train.shape,  y_train.shape)
print ('Test set:', X_test.shape,  y_test.shape)
```

```
Train set: (160, 7) (160,)
Test set: (40, 7) (40,)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
LR = LogisticRegression(C=0.01, solver='liblinear').fit(X_train,y_train)
LR
```

```
LogisticRegression(C=0.01, solver='liblinear')
```

```
yhat = LR.predict(X_test)
yhat
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0])
```

```
yhat_prob = LR.predict_proba(X_test)
yhat_prob
```

```
array([[0.54132919, 0.45867081],
       [0.60593357, 0.39406643],
       [0.56277713, 0.43722287],
       [0.63432489, 0.36567511],
       [0.56431839, 0.43568161],
       [0.55386646, 0.44613354],
       [0.52237207, 0.47762793],
       [0.60514349, 0.39485651],
       [0.41069572, 0.58930428],
       [0.6333873 , 0.3666127 ],
       [0.58068791, 0.41931209],
       [0.62768628, 0.37231372],
       [0.47559883, 0.52440117],
       [0.4267593 , 0.5732407 ],
       [0.66172417, 0.33827583],
       [0.55092315, 0.44907685],
       [0.51749946, 0.48250054],
       [0.485743  , 0.514257  ],
       [0.49011451, 0.50988549],
       [0.52423349, 0.47576651],
       [0.61619519, 0.38380481],
       [0.52696302, 0.47303698],
       [0.63957168, 0.36042832],
       [0.52205164, 0.47794836],
       [0.50572852, 0.49427148],
       [0.70706202, 0.29293798],
       [0.55266286, 0.44733714],
       [0.52271594, 0.47728406],
       [0.51638863, 0.48361137],
       [0.71331391, 0.28668609],
       [0.67862111, 0.32137889],
       [0.50896403, 0.49103597],
       [0.42348082, 0.57651918],
       [0.71495838, 0.28504162],
       [0.59711064, 0.40288936],
```

```
from sklearn.metrics import jaccard_score
jaccard_score(y_test, yhat,pos_label=0)
```

```
0.7058823529411765
```

```
from sklearn.metrics import classification_report, confusion_matrix
import itertools
def plot_confusion_matrix(cm, classes,
                          normalize=False,
                          title='Confusion matrix',
                          cmap=plt.cm.Blues):
    """
    This function prints and plots the confusion matrix.
    Normalization can be applied by setting `normalize=True`.
    """
    if normalize:
        cm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
        print("Normalized confusion matrix")
    else:
        print('Confusion matrix, without normalization')

    print(cm)

    plt.imshow(cm, interpolation='nearest', cmap=cmap)
    plt.title(title)
    plt.colorbar()
    tick_marks = np.arange(len(classes))
    plt.xticks(tick_marks, classes, rotation=45)
    plt.yticks(tick_marks, classes)

    fmt = '.2f' if normalize else 'd'
    thresh = cm.max() / 2.
    for i, j in itertools.product(range(cm.shape[0]), range(cm.shape[1])):
        plt.text(j, i, format(cm[i, j], fmt),
                 horizontalalignment="center",
                 color="white" if cm[i, j] > thresh else "black")

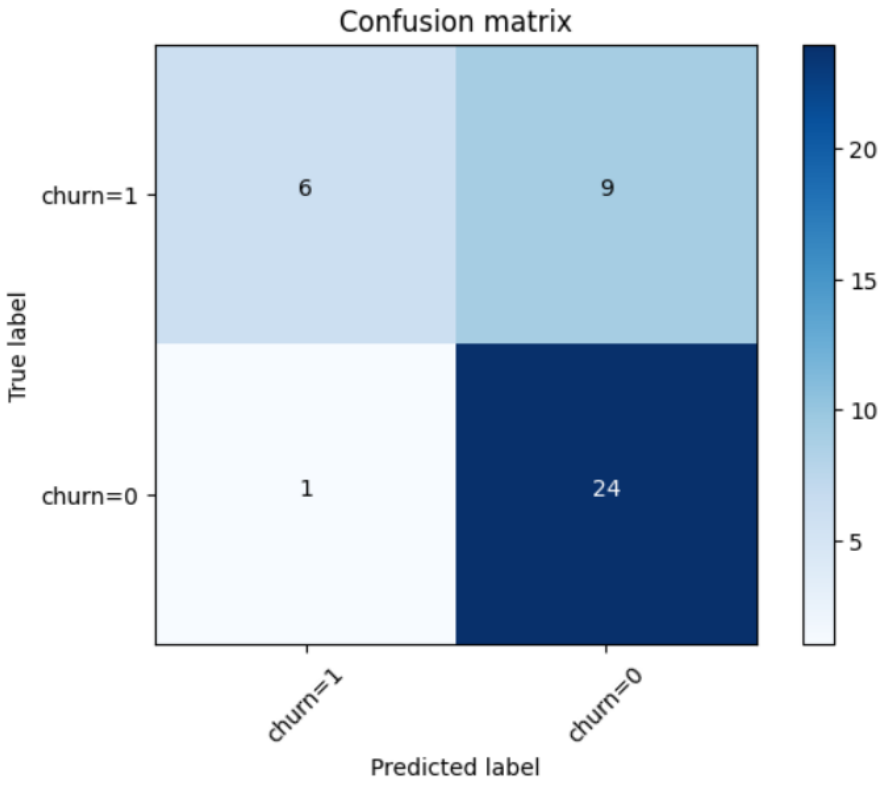
    plt.tight_layout()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
print(confusion_matrix(y_test, yhat, labels=[1,0]))
```

```
[[ 6  9]
 [ 1 24]]
```

```
# Compute confusion matrix
cnf_matrix = confusion_matrix(y_test, yhat, labels=[1,0])
np.set_printoptions(precision=2)
#conf matrix = eval out dr model predict thdp model sebenarnya
#proses training -> model. Dataset -> train & test (sbnrnya udah tau outnya, cm skrg diilangkan label out nya)
#test set diuji ke model

# Plot non-normalized confusion matrix
plt.figure()
plot_confusion_matrix(cnf_matrix, classes=['churn=1','churn=0'],normalize= False,  title='Confusion matrix')
```

```
Confusion matrix, without normalization
[[ 6  9]
 [ 1 24]]
```




```
print (classification_report(y_test, yhat))
```

```
              precision    recall  f1-score   support

           0       0.73      0.96      0.83        25
           1       0.86      0.40      0.55        15

    accuracy                           0.75        40
   macro avg       0.79      0.68      0.69        40
weighted avg       0.78      0.75      0.72        40
```

```
from sklearn.metrics import log_loss
log_loss(y_test, yhat_prob)
```

```
0.6017092478101185
```

```
# write your code here

LR2 = LogisticRegression(C=0.01, solver='sag').fit(X_train,y_train)
yhat_prob2 = LR2.predict_proba(X_test)
print ("LogLoss: : %.2f" % log_loss(y_test, yhat_prob2))
```

```
LogLoss: : 0.61
```

c) Analisis

Logistic regression merupakan algoritma klasifikasi yang digunakan untuk mencari hubungan antara input diskrit/kontinu dengan probabilitas hasil output diskrit tertentu. Pada program praktikum ini, tipe logistic regression yang digunakan adalah binary logistic regression yaitu tipe yang memiliki dua output saja dan mengklasifikasikan ke dua kelas yang berbeda. Dalam hal ini, dihasilkan output churn dan tidak churn. Program ini digunakan untuk konteks klasifikasi memprediksi pelanggan suatu perusahaan akan churn (berhenti berlangganan) atau tidak.

Serupa dengan linear regression, tahap awal yang dilakukan adalah membaca data program dari file ChurnData.csv menggunakan library pandas yang kemudian digunakan sebagai dataset untuk membangun model regresi logistik. Selanjutnya fitur-fitur yang terdapat pada dataset diseleksi dengan memilih hanya kolom-kolom tertentu dengan asumsi bahwa fitur tersebut akan digunakan untuk memprediksi kecenderungan churn seorang pelanggan.

Program melakukan konversi label dengan mengubah tipe data secara spesifik pada kolom ‘churn’ menjadi integer lalu semua fitur yang dipilih dinormalisasi menjadi skala standar melalui metode penskalaan. Dengan langkah ini, maka semua fitur akan ada pada skala yang serupa sehingga model menjadi memiliki kinerja yang lebih baik. Dataset juga kemudian dibagi menjadi dua yaitu dataset untuk pelatihan dan untuk pengujian yang kemudian diketahui sebagai train dan test. Digunakan sebanyak 20% data dari data keseluruhan untuk menjadi test set, sedangkan 80% data untuk menjadi train set.

Model regresi dilatih menggunakan dataset train menggunakan parameter C = 0.01 dan solver liblinear dari class LogisticRegression dengan pemanggilan method fit. class tersebut diambil melalui library LogisticRegression yang berada pada package sklearn.linear_model, serta library confusion_matrix yang diambil dari package sklearn.metrics. Model kemudian memprediksi dataset train melalui perintah LR memanggil method predict(X_test) yang disimpan pada variabel yhat. Sedangkan untuk mempediksi probabilitasnya digunakan pemanggilan method predict_proba yang disimpan pada yhat_prob. Dengan ini akan dihasilkan dua array, yaitu

array hasil prediksi antara nilai 0 dan 1 serta array probablilitas dari rentang nilai 0 hingga 1.

Untuk menyempurnakan model, dilakukan evaluasi model melalui perhitungan akurasi, presisi, recall, f1-score, dan jaccard score. Jaccard merupakan ukuran dari perpotongan yang dibagi dengan ukuran gabungan dua label. Jika keseluruhan label yang diprediksi pada suatu sampel benar-benar sesuai dengan kumpulan label sebenarnya maka keakuratannya adalah 1. Pada program ini dihasilkan jaccard score senilai 0.7 yang mendekati nilai 1, artinya masih ada error yang terjadi pada pemodelan mesinnya.

Program ini juga menyertakan fungsi plot_confusion_matrix untuk menghasilkan matrix confusion yang membantu visualisasi sejauh mana model memprediksi dengan benar dan dimana kesalahan klasifikasi yang terjadi. Nilai 6 pada confusion matrix adalah true positif yaitu kondisi ketika model dengan benar memprediksi contoh sebagai kelas positive, nilai 24 adalah true negative yaitu kondisi ketika model dengan benar memprediksi contoh sebagai kelas negative, nilai 1 adalah false positive yaitu kondisi ketika model keliru memprediksi contoh sebagai kelas positif yang seharusnya kelas negatif (kesalahan tipe I), dan nilai 9 adalah false negative yaitu kondisi ketika model keliru memprediksi contoh sebagai kelas negatif yang seharusnya kelas positif (kesalahan tipe II).

d) F1-Score

F1-score merupakan salah satu metrik evaluasi yang digunakan dalam konteks klasifikasi logistik tipe biner. F1-score menggabungkan precision dan recall untuk memberikan gambaran yang lebih komprehensif tentang kinerja model. Precision merupakan rasio antara jumlah positif sejati yang berhasil diprediksi oleh model terhadap total prediksi positif (positif sejati dan positif palsu). Dalam rumus, presisi dapat dihitung sebagai: Prec = $\frac{TP}{TP + FP}$. Semakin kecil False Positive (FP) maka semakin besar precission yang dihasilkan.

Sementara itu, recall (sensitivitas) merupakan rasio antara jumlah positif sejati yang berhasil diprediksi oleh model terhadap total positif sejati yang sebenarnya ada. Dalam rumus, recall dapat dihitung sebagai: Recall=

$\frac{TP}{TP + FN}$. Semakin kecil nilai False Negative (FN) maka semakin besar recall yang dihasilkan.

F1-score adalah harmonik mean dari presisi dan recall, yang memberikan bobot yang seimbang kepada keduanya. F1-score adalah metrik untuk menghindari kesalahan palsu positif (FP) dan palsu negatif (FN) dalam kasus keseimbangan antara presisi dan recall. Dalam rumus, f1-score dihitung sebagai: F1-score = 2 * $\frac{Prec*Recall}{Prec+Recall}$. Nilai yang dihasilkan adalah berkisar antara 0 hingga 1. Ketika nilai skor yang dihasilkan tinggi, maka kinerja model lebih baik dalam hal presisi dan sensitivitas yang seimbang.